

DOCKER COMPOSE: ORQUESTRAÇÃO DE MÚLTIPLOS CONTAINERS

Desenvolvimento Full Stack

1: INTRODUÇÃO

O que é Docker Compose?

Docker Compose é uma ferramenta que permite definir e executar aplicações compostas por múltiplos containers Docker por meio de um único arquivo de configuração em formato YAML. Em vez de gerenciar cada container separadamente com comandos `docker run`, declara-se todos os serviços, redes e volumes em um arquivo `docker-compose.yml` e utiliza comandos simples como `docker-compose up` para colocar tudo em funcionamento de forma coordenada.

Por que usar Docker Compose?

Reprodutibilidade: Um único arquivo garante que todos os desenvolvedores do time executem exatamente o mesmo ambiente.

Simplificação: Elimina a necessidade de decorar longas sequências de parâmetros de linha de comando.

Documentação viva: O arquivo `docker-compose.yml` funciona como documentação do projeto, de modo que qualquer pessoa vê quais serviços são necessários, portas, variáveis de ambiente e dependências.

Desenvolvimento rápido: Levantar um ambiente completo é muito mais simples.

Docker vs Docker Compose

Aspecto	Docker	Docker Compose
Escopo	Gerencia um container por vez	Orquestra múltiplos containers
Comando	<code>docker run</code> (longo e complexo)	<code>docker-compose up</code> (simples)
Arquivo config	Não há (tudo via CLI)	<code>docker-compose.yml</code> (declarativo)
Rede entre containers	Requer configuração manual	Automática
Caso de uso	Container isolado	Aplicação completa (API + BD + Cache)

Casos de Uso

- Aplicações web full stack: Front-end, back-end, banco de dados em um único comando.
- Microsserviços: Múltiplos serviços especializados rodando juntos.
- Ambiente de desenvolvimento: Simular produção localmente.
- CI/CD: Testar a aplicação completa em pipelines automatizados.
- Onboarding: Novos desenvolvedores rodando `docker-compose up` e tendo tudo pronto.

Componentes Principais

Serviços: Cada container lógico da aplicação (API, banco de dados, cache, etc.).

Redes: Permitem a comunicação entre serviços usando nomes de serviço como hostnames (ex: `mysql`, `redis`).

Volumes: Armazenam dados de forma persistente, sobrevivendo ao ciclo de vida dos containers.

Portas: Mapeamento entre a máquina host (seu PC) e os containers (ex: `3000:3000` significa porta 3000 do host vai para porta 3000 do container).

2: FUNDAMENTOS

Arquitetura do Docker Compose

Quando é executado `docker-compose up` ocorre:

1. Leitura do arquivo: Docker Compose lê o arquivo `docker-compose.yml`.
2. Criação de rede: Cria uma rede bridge conectando todos os serviços.
3. Download de imagens: Faz pull das imagens necessárias (`mysql`, `node`, `redis`, etc.).
4. Build de imagens customizadas: Se há um `Dockerfile`, constrói a imagem (ex: uma aplicação `Node.js`).
5. Criação de volumes: Prepara os volumes para persistência de dados.
6. Inicialização de containers: Inicia os serviços na ordem correta de dependências.
7. Logs em tempo real: Exibe os logs de todos os containers simultaneamente.

Estrutura do `docker-compose.yml`

```
version: '3.8'

services:
  meu-servico:
    image: node:18-alpine
    container_name: meu-container
    ports:
      - "3000:3000"
    environment:
      DB_HOST: mysql
      DB_USER: root
    volumes:
      - ./app:/usr/src/app
    networks:
      - minha-rede
    depends_on:
      - mysql

mysql:
```

```

image: mysql:8.0
# ... outras propriedades

networks:
  minha-rede:
    driver: bridge

volumes:
  dados-mysql:

```

Propriedades Principais de um Serviço

image: Nome e tag da imagem Docker (ex: `node:18-alpine`, `mysql:8.0`).

container_name: Nome do container para facilitar referência (ex: `api-app`).

ports: Mapeamento de porta no formato "HOST:CONTAINER" (ex: "3000:3000").

Se quiser apenas expor internamente, omita esta propriedade.

environment: Variáveis de ambiente.

Pode ser lista (`- CHAVE=valor`) ou mapa (`CHAVE: valor`).

volumes: Persistência de dados.

- Volume nomeado: `nome-volume:/caminho/no/container`
- Bind mount: `./caminho/local:/caminho/no/container`

networks: Redes às quais o serviço se conecta.

depends_on: Lista de serviços que devem ser iniciados antes deste.

Observação: `depends_on` aguarda a inicialização, não a "prontidão" do serviço. Para isso, adicione `retry` ou `health check` na aplicação.

restart: Política de reinicialização (`no`, `always`, `on-failure`, `unless-stopped`).

command: Comando customizado para executar (sobrescreve `CMD` do `Dockerfile`).

Ciclo de Vida

Comando	O que faz	Dados persistem?
<code>docker-compose up</code>	Cria e inicia todos os containers	Sim (volumes)
<code>docker-compose down</code>	Para e remove containers, redes	Sim (volumes)
<code>docker-compose down -v</code>	Para, remove containers E volumes	Não — dados perdidos!
<code>docker-compose start</code>	Inicia containers parados	Sim
<code>docker-compose stop</code>	Para containers sem remover	Sim
<code>docker-compose pause</code>	Suspende processos (não recomendado)	Sim
<code>docker-compose restart</code>	Reinicia containers	Sim

3: ATIVIDADE PRÁTICA

Stack: API Node.js + MySQL + Redis

Node.js + Express: API simples que conecta a banco de dados e cache.

MySQL 8.0: Banco de dados relacional.

Redis: Cache em memória.

phpMyAdmin: Interface gráfica para administrar MySQL.

3.1 PREPARAÇÃO DO AMBIENTE

Passo 1: Verificar WSL2 e Hyper-V

Abra o PowerShell (Windows + X) e execute:

```
wsl --status
```

Se exibir "WSL 2: Habilitado", pule para o Passo 2.

Se exibir "WSL 2 é necessário" ou erro, execute:

```
wsl --install
```

Aguarde a instalação e reinicie o computador. Após reiniciar, execute novamente `wsl --status` para confirmar.

Passo 2: Baixar e Instalar Docker Desktop

1. Acesse: <https://www.docker.com/products/docker-desktop/>
2. Clique em "Download for Windows"
3. Execute o instalador (Docker Desktop Installer.exe)
4. Na tela de instalação, marque a opção "Use WSL 2 instead of Hyper-V"
5. Clique em "Install"
6. Ao final, clique em "Close and restart"
7. O computador reiniciará e Docker Desktop será iniciado automaticamente

Passo 3: Verificar se Docker está funcionando

Abra um novo PowerShell e execute:

```
docker --version
```

Saída esperada: Docker version 26.1.4, build xxxxxx

Passo 4: Criar pasta do projeto

```
mkdir C:\projeto-fullstack
```

```
cd C:\projeto-fullstack
```

3.2 ESTRUTURA DO PROJETO

Dentro de `C:\projeto-fullstack`, crie as seguintes pastas e arquivos:

```
projeto-fullstack/
├─ docker-compose.yml
├─ app/
│   ├─ Dockerfile
│   ├─ package.json
│   ├─ server.js
│   └─ .dockerignore
├─ mysql-init/
│   └─ init.sql
└─ nginx/
    └─ nginx.conf
```

Para criar as pastas, no PowerShell execute:

```
mkdir app
mkdir mysql-init
mkdir nginx
```

3.3 CRIANDO OS ARQUIVOS

Arquivo 1: docker-compose.yml

Localização: `C:\projeto-fullstack\docker-compose.yml`

Conteúdo:

```
version: '3.8'

services:
  api-nodejs:
    build: ./app
    container_name: api-app
    ports:
      - "3000:3000"
    environment:
      - DB_HOST=mysql
      - DB_USER=root
      - DB_PASSWORD=rootpassword
      - DB_NAME=meubanco
```

```
- REDIS_HOST=redis
- REDIS_PORT=6379
depends_on:
  - mysql
  - redis
networks:
  - app-network
volumes:
  - ./app:/usr/src/app
  - /usr/src/app/node_modules
```

```
mysql:
  image: mysql:8.0
  container_name: mysql-db
  restart: always
  environment:
    MYSQL_ROOT_PASSWORD: rootpassword
    MYSQL_DATABASE: meubanco
  ports:
    - "3306:3306"
  volumes:
    - mysql-data:/var/lib/mysql
    - ./mysql-init:/docker-entrypoint-initdb.d
  networks:
    - app-network
```

```
redis:
  image: redis:7-alpine
  container_name: redis-cache
  restart: always
  ports:
    - "6379:6379"
  networks:
    - app-network
```

```
phpmyadmin:
  image: phpmyadmin/phpmyadmin:latest
  container_name: phpmyadmin-ui
  restart: always
  ports:
    - "8081:80"
  environment:
    PMA_HOST: mysql
    PMA_USER: root
    PMA_PASSWORD: rootpassword
  depends_on:
    - mysql
  networks:
```

```
- app-network
```

```
networks:  
  app-network:  
    driver: bridge
```

```
volumes:  
  mysql-data:
```

Arquivo 2: app/Dockerfile

Localização: C:\projeto-fullstack\app\Dockerfile

Como criar:

1. Entre na pasta **app**
2. Clique com botão direito, "Novo" → "Documento de Texto"
3. Renomeie para **Dockerfile** (sem extensão .txt, confirme a mudança)
4. Cole o conteúdo abaixo

Conteúdo:

```
FROM node:18-alpine  
  
WORKDIR /usr/src/app  
  
COPY package*.json ./  
  
RUN npm install  
  
COPY . .  
  
EXPOSE 3000  
  
CMD ["node", "server.js"]
```

Arquivo 3: app/package.json

Localização: C:\projeto-fullstack\app\package.json

Conteúdo:

```
{  
  "name": "projeto-fullstack",  
  "version": "1.0.0",  
  "description": "API Node.js com MySQL e Redis",
```

```

"main": "server.js",
"scripts": {
  "start": "node server.js",
  "dev": "nodemon server.js"
},
"dependencies": {
  "express": "^4.18.2",
  "mysql2": "^3.9.0",
  "redis": "^4.6.12",
  "dotenv": "^16.3.1"
},
"devDependencies": {
  "nodemon": "^3.0.2"
}
}

```

Arquivo 4: app/server.js

Localização: C:\projeto-fullstack\app\server.js

Conteúdo:

```

const express = require('express');
const mysql = require('mysql2/promise');
const redis = require('redis');
require('dotenv').config();

const app = express();
const PORT = 3000;

// Configuração MySQL
const dbConfig = {
  host: process.env.DB_HOST || 'mysql',
  user: process.env.DB_USER || 'root',
  password: process.env.DB_PASSWORD || 'rootpassword',
  database: process.env.DB_NAME || 'meubanco',
};

// Configuração Redis
const redisClient = redis.createClient({
  url: `redis://${process.env.REDIS_HOST || 'redis'}:${process.env.REDIS_PORT || 6379}`
});

redisClient.on('error', (err) => console.error('Redis error:', err));

```



```

async function connectRedis() {
  await redisClient.connect();
  console.log('Conectado ao Redis');
}

connectRedis();

// Rota de teste: conexão MySQL
app.get('/db-check', async (req, res) => {
  try {
    const connection = await mysql.createConnection(dbConfig);
    const [rows] = await connection.execute('SELECT 1 + 1 AS resultado');
    await connection.end();
    res.json({
      status: 'ok',
      resultado: rows[0].resultado,
      database: dbConfig.database
    });
  } catch (err) {
    res.status(500).json({ status: 'erro', mensagem: err.message });
  }
});

// Rota de teste: conexão Redis
app.get('/redis-check', async (req, res) => {
  try {
    await redisClient.set('teste', 'conexao-ok');
    const valor = await redisClient.get('teste');
    res.json({ status: 'ok', redis: valor });
  } catch (err) {
    res.status(500).json({ status: 'erro', mensagem: err.message });
  }
});

// Rota principal
app.get('/', (req, res) => {
  res.send('API Node.js rodando! Acesse /db-check ou /redis-check para testar.');
```

```

});

app.listen(PORT, () => {
  console.log(`Servidor rodando na porta ${PORT}`);
});

```

Arquivo 5: app/.dockerignore

Localização: C:\projeto-fullstack\app\.dockerignore

Conteúdo:

```
node_modules
npm-debug.log
.git
.gitignore
.env
```

Arquivo 6: mysql-init/init.sql

Localização: C:\projeto-fullstack\mysql-init\init.sql

Conteúdo:

```
CREATE TABLE IF NOT EXISTS usuarios (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(100) NOT NULL,
  email VARCHAR(100) NOT NULL UNIQUE,
  data_cadastro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO usuarios (nome, email) VALUES
('João Silva', 'joao@email.com'),
('Maria Santos', 'maria@email.com');
Salve.
```

3.4 EXECUTANDO O PROJETO

1: Abrir o Windows PowerShell

2: Navegar até a pasta do projeto

```
cd C:\projeto-fullstack
```

3: Executar o Docker Compose

```
docker-compose up
```

Resultado esperado:

- Docker irá baixar as imagens (node, mysql, redis, phpmyadmin)
- Construirá a imagem da sua aplicação Node.js
- Iniciará todos os 4 containers

Exibirá logs em tempo real

A primeira execução pode levar 2-3 minutos. Aguarde até ver mensagens como:

```
api-app | Servidor rodando na porta 3000
mysql-db | ready for connections
redis-cache | Ready to accept connections
```

4: Verificar se tudo está rodando

Abra um segundo PowerShell (ATENÇÃO: **não feche o primeiro**) e execute:

```
docker ps
```

Saída esperada:

CONTAINER ID	IMAGE	STATUS	PORTS
abc123def456	projeto-fullstack-app	Up 2 minutes	0.0.0.0:3000->3000/tcp
def456ghi789	mysql:8.0	Up 2 minutes	0.0.0.0:3306->3306/tcp
ghi789jkl012	redis:7-alpine	Up 2 minutes	0.0.0.0:6379->6379/tcp
jkl012mno345	phpmyadmin/phpmyadmin	Up 2 minutes	0.0.0.0:8081->80/tcp

5: Acessar a aplicação

Abra seu navegador e acesse:

```
http://localhost:3000
```

Resultado esperado:

API Node.js rodando! Acesse /db-check ou /redis-check para testar.

6: Testar conexão com MySQL

No mesmo navegador, acesse:

```
http://localhost:3000/db-check
```

Resultado esperado:

```
{
  "status": "ok",
  "resultado": 2,
  "database": "meubanco"
}
```

Isso significa que a aplicação Node.js conseguiu conectar ao MySQL e executar uma query!

7: Testar conexão com Redis

Acesse:

`http://localhost:3000/redis-check`

Resultado esperado:

```
{
  "status": "ok",
  "redis": "conexao-ok"
}
```

Significa que a aplicação está se comunicando com Redis.

8: Acessar o phpMyAdmin

Acesse:

`http://localhost:8081`

Para fazer login:

- Usuário: root
- Senha: rootpassword

Será possível verificar no banco de dados `meubanco` uma tabela `usuarios` contendo os 2 registros inseridos via `init.sql`.

3.5 DEPURANDO E EXPLORANDO

Ver logs de um serviço específico

No powershell:

```
docker logs api-app
docker logs mysql-db
docker logs redis-cache
```

Acompanhar logs em tempo real

```
docker logs -f api-app
```

Pressione Ctrl+C para sair.

Executar comandos dentro de um container

```
docker exec -it api-app sh
```

Dentro do container, é possível:

Listar arquivos: `ls`

Ver variáveis de ambiente: `env` / `grep DB`

Ver versão do Node: `node --version`

Para sair, digite `exit`.

Verificar redes criadas

No powershell:

```
docker network ls
docker network inspect projeto-fullstack_app-network
```

Será possível verificar todos os containers conectados à mesma rede, podendo se comunicar pelo nome (ex: `mysql`, `redis`).

Inspecionar volumes

No powershell:

```
docker volume ls
docker volume inspect projeto-fullstack_mysql-data
```

O volume `mysql-data` armazena todos os dados do MySQL. Ele persiste mesmo após `docker-compose down`.

3.6 ENCERRANDO O PROJETO

Parar containers sem remover dados

```
docker-compose stop
```

Os containers param, mas as redes e volumes permanecem. Execute `docker-compose start` para retomar.

Parar e remover containers, redes (mas manter volumes)

```
docker-compose down
```

Use isso no final de cada sessão de desenvolvimento.

Atenção: Remover tudo, incluindo dados do MySQL

```
docker-compose down -v
```

Cuidado: O flag `-v` remove os volumes. Todos os dados do MySQL serão perdidos permanentemente. Use apenas se tiver certeza!

4: BOAS PRÁTICAS

Versionamento de Imagens

✗ Evitar:

```
image: mysql
image: node
image: redis
```

☑ Fazer:

```
image: mysql:8.0
image: node:18-alpine
image: redis:7-alpine
```

Por quê: Tags específicas garantem que a mesma imagem seja usada em todos os ambientes. `latest` pode mudar inesperadamente e quebrar sua aplicação.

Variáveis de Ambiente com `.env`

Nunca coloque senhas e dados sensíveis diretamente no `docker-compose.yml`.

Crie um arquivo `.env` na raiz do projeto:

```
DB_PASSWORD=minha_senha_super_secreta
MYSQL_ROOT_PASSWORD=outra_senha
REDIS_PASSWORD=redis_senha
```

No `docker-compose.yml`, referencie com `${VARIABLE}`:

```
environment:
  MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
  DB_PASSWORD: ${DB_PASSWORD}
```

Adicione `.env` ao `.gitignore` para não versionar senhas:

```
.env
node_modules/
.git/
```

Segurança

- Nunca use a senha padrão `rootpassword` em produção
- Não exponha portas desnecessariamente (ex: não exponha a porta 3306 do MySQL para a internet)
- Use volumes para dados sensíveis (senhas, chaves)
- Mantenha imagens atualizadas regularmente

Organização de Arquivos

Mantenha uma estrutura clara:

- Um `docker-compose.yml` por projeto

- Para ambientes múltiplos (dev, staging, prod), crie: `docker-compose.yml`, `docker-compose.dev.yml`, `docker-compose.prod.yml`
- Use `docker-compose -f docker-compose.prod.yml up` para ambiente específico

Documentação

Crie um `README.md` no projeto:

```
# Projeto Full Stack com Docker Compose

## Requisitos
- Docker Desktop 4.0+
- Windows 10/11 com WSL2

## Como executar
1. `cd projeto-fullstack`
2. `docker-compose up`
3. Acesse http://localhost:3000

## Serviços
- API: http://localhost:3000
- phpMyAdmin: http://localhost:8081
- MySQL: localhost:3306
- Redis: localhost:6379

## Logs
`docker logs -f api-app`

## Parar
`docker-compose down`
```

SEÇÃO 5: TROUBLESHOOTING

WSL2 não está habilitado

Erro: WSL 2 installation is incomplete. Please restart your computer.

Solução:

```
wsl --install
```

Reinicie o computador.

Porta 3000 já em uso

Erro: bind: address already in use

Solução 1: Encontre o processo usando a porta

```
netstat -ano | findstr :3000
```

Anote o PID. Para encerrá-lo:

```
taskkill /PID 12345 /F
```

Solução 2: Use outra porta no `docker-compose.yml`:

```
ports:
  - "3001:3000"  # mudou de 3000 para 3001
```

Erro de permissão ao montar volumes no Windows

Erro: Permission denied

Solução: No Docker Desktop, vá em:

1. Clique no ícone da baleia (bandeja do sistema)
2. Settings → Resources → File Sharing
3. Adicione a pasta `C:\projeto-fullstack`
4. Clique em "Apply & Restart"

Container sai com "Exit code 1"

Erro: Container exited with code 1

Solução: Ver logs do container

```
docker logs api-app
```

Procure pela linha de erro. Causas comuns:

- `npm install` falhou: verifique `package.json`
- Dependência faltando: certifique-se que todas as dependências estão em `package.json`
- Volume conflitando: remova `node_modules` local e deixe o container instalá-lo

Erro de conexão entre containers

Erro: Cannot reach mysql from api-nodejs

Solução: Os containers já estão na mesma rede. O problema é `depends_on` + aplicação tentando conectar imediatamente.

No `server.js`, adicione `retry`:

```
async function conectarMySQL() {
  let tentativas = 5;
  while (tentativas > 0) {
    try {
      const connection = await mysql.createConnection(dbConfig);
      console.log('Conectado ao MySQL');
      return connection;
    } catch (err) {
```



```
tentativas--;  
console.log(`Tentando novamente... (${tentativas})`);  
await new Promise(r => setTimeout(r, 2000));  
}  
}  
throw new Error('Não conseguiu conectar ao MySQL');  
}
```

O que aprendemos:

- O que é Docker Compose e por que usar
- Como estruturar um `docker-compose.yml`
- Como criar e executar um projeto com múltiplos containers
- Como depurar e explorar containers
- Boas práticas e como resolver problemas comuns

Próximos passos:

1. Customize o projeto: Adicione autenticação, mais endpoints, etc.
2. Adicione Nginx: Configure um proxy reverso no projeto
3. Deploy: Suba o projeto em um servidor (AWS, DigitalOcean, etc.)
4. CI/CD: Integre com GitHub Actions para testes automatizados